ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

A Detour on Scala: Classes, Companion Objects, and Extension Methods

The lectures on datatype design and abstract datatypes make use of several Scala features that have not been introduced systematically so far: classes, companion objects, and extension methods. This detour provides a self-contained presentation of these features.

Classes

Just as in Java, Scala classes are the blueprint (and the type) for objects: once we define a class, we can create objects from it using the keyword `new`. A class definition contains *fields* — declared as modifiable `var`iables or immutable `val`ues, holding the data of the objects of the class — and *methods*, introduced with the keyword `def`, containing executable code that has access to the fields.

Here is a class for a stack of integers:

```
class IntStack(val elements: List[Int]):
  def push(x: Int): IntStack = new IntStack(x :: elements)

  def top: Int = elements match
    case Nil    => sys.error("empty stack")
    case x :: _ => x

  def pop: IntStack = elements match
    case Nil    => sys.error("empty stack")
    case _ :: t => new IntStack(t)

  def isEmpty: Boolean = elements.isEmpty
```

This `IntStack` class has one field (`elements`) and four methods (`push`, `top`, `pop`, `isEmpty`). Stacks are constructed with `new`:

```
val s = new IntStack(Nil)
val s1 = s.push(1).push(2).push(3)
s1.top      // 3
```

Primary constructors. Unlike many other OO languages, the class definition includes the signature of the primary constructor directly in the class header — `(val elements: List[Int])`. Parameters annotated with `val` (or `var`) become fields of the class, accessible from outside. Parameters without `val` or `var` are private to the class — visible only from within. Class members declared inside the class body are public by default; the `private` access modifier hides them from outside.

To make a field private, it must be declared inside the class rather than via a constructor parameter:

```
class IntStack(elements: List[Int]):        // elements is private
  private val data: List[Int] = elements    // also private
  def isEmpty: Boolean = data.isEmpty
```

Side remark. Throughout this course, we insist on functional (stateless / immutable) data structures. Scala provides full support for stateful objects, whose state is defined by `var`-tagged fields (or `var` constructor parameters) just as in Java, but we do not use this style.

Singleton objects

Beyond classes, Scala provides **singleton objects** — declared with the `object` keyword. The declaration `object Counter` introduces a unique, automatically created instance of a fresh anonymous class; there is no way to make further instances of it. Singleton objects play the role that *static* members play in Java: they are the natural place to host functions, constants, and definitions that do not belong to any particular instance of a class.

```
object MathUtils:
  val pi: Double = 3.14159
  def square(x: Double): Double = x * x
  def cube(x: Double): Double = x * x * x

MathUtils.pi          // 3.14159
MathUtils.square(4)   // 16.0
```

`MathUtils` is itself a value (the unique instance of an anonymous class), accessible by name. We can pass it around, store it in variables, and call its members — but we cannot construct a second

`MathUtils` .

Companion objects

A *companion object* is a singleton object that shares its name with another type/class definition in the same file. The two — the type and the companion object — are linked, in that they can access each other's private members. This makes the companion object the natural place to host definitions that conceptually "belong to" the type but are not tied to a specific value of it: factory methods, constants, helper functions.

Companion objects host a variety of definitions, including methods, factory functions, and constants. One paradigmatic example is the `apply` **factory method**. A *factory method* is a function that constructs and returns a value of a given type: it plays the role of a constructor, but as a regular method rather than a special language construct. Why does this matter? Because Scala gives `apply` a privileged status: when the compiler encounters `Foo(args)` and `Foo` is the name of a type with a companion object defining `apply` , the compiler automatically rewrites the call as `Foo.apply(args)` . From the client's point of view, `Foo(args)` looks like an invocation of `Foo` 's constructor, but under the hood, it is a call to the `apply` method in the companion object, which then decides how to construct the result. The factory method gives the type author full control over what "constructing a value" means, while the client sees a uniform constructor-like syntax.

For our `IntStack` , a companion object can provide an `apply` method that constructs a stack from a sequence of elements, plus an `empty` value:

```
class IntStack(val elements: List[Int]):
  def push(x: Int): IntStack = new IntStack(x :: elements)
  def top: Int = elements.head
  def pop: IntStack = new IntStack(elements.tail)
  def isEmpty: Boolean = elements.isEmpty

object IntStack:
  def apply(xs: Int*): IntStack = new IntStack(xs.toList)
  // . . .
```

Now clients can construct stacks naturally:

```
val s1 = IntStack(1, 2, 3)        // calls IntStack.apply(1, 2, 3)
val s2 = IntStack()
val s3 = IntStack()
```

A point worth clarifying: although the companion object `IntStack` is itself a singleton (there is only one), the factory method `apply` creates new instances of the *class* `IntStack`, of which there can be arbitrarily many. The "singleton" qualifier applies to the companion object as a *container of definitions*, not to the values it produces.

Companion objects are not exclusive to classes. Any type definition can have one — a class, a trait, an `enum`, or an *opaque type alias* (which we introduce in the encapsulation lecture). In all these cases, the companion object hosts factory methods and other definitions associated with the type, with the same privileged access to private members. Plain (non-opaque) type aliases — like `type Foo = Int` — typically do not have companion objects, because the alias is just another name for an existing type and shares its existing companion (if any).

Case classes and traits

The `IntStack` class above works, but its plain-class form has limitations: pattern matching is not available, structural equality must be implemented by hand, the boilerplate of `new` cannot be avoided without writing a companion object explicitly. Scala provides **case classes** as a more convenient way to define immutable data:

```
case class Circle(radius: Double)
case class Rectangle(base: Double, height: Double)
```

Case classes come with several features automatically:

- **A factory method** in an automatically generated companion object, so `Circle(3.0)` works without `new`.
- **Pattern matching** support, so case class instances can be destructured via patterns.
- **Structural equality**: `equals` (and the `==` operator that delegates to it) compares instances field by field, so two case class values are equal whenever their corresponding fields are equal.
- **Useful default methods**: `toString`, `hashCode`, and a `copy` method for creating modified versions.
- **Public fields** by default: all constructor parameters are implicitly `val`, so they become immutable fields.

To group related case classes under a common supertype, Scala provides **sealed traits**:

```
sealed trait Shape
case class Circle(radius: Double) extends Shape
case class Rectangle(base: Double, height: Double) extends Shape
```

A `trait` is similar to a Java interface, but it can also include method implementations. The `sealed` modifier means that no further subtypes can be defined outside this file. Sealing helps the type checker verify the exhaustiveness of pattern matches: if a `match` expression on a `Shape` covers `Circle` and `Rectangle`, the compiler knows there are no other cases to worry about.

In Scala 3, the equivalent more concise form uses `enum`:

```
enum Shape:
  case Circle(radius: Double)
  case Rectangle(base: Double, height: Double)
```

This desugars to exactly the `sealed trait` + case classes form above.

Extension methods

Sometimes we want to add a method to a type *after* the type is defined — perhaps because we don't own the type's source code, or because we want to keep the type's definition focused on its core operations and add convenience methods elsewhere.

Scala provides **extension methods** for this purpose. The syntax declares a parameter (the receiver) and a list of methods that take that receiver implicitly:

```
case class Circle(x: Double, y: Double, radius: Double)

extension (c: Circle)
  def circumference: Double = c.radius * math.Pi * 2
```

Extension methods are invoked with the standard dot notation:

```
val circle = Circle(0, 0, 1)
circle.circumference     // 6.283...
```

The extension reads as: "for any value `c` of type `Circle`, the method `circumference` is available, and inside its body `c` refers to the receiver."

Operators. Because Scala treats operators as ordinary method names, the extension syntax can also define new operators on existing types:

```
extension (x: Int)
  def **(n: Int): Int =                    // Int does not have ** by default
    if n == 0 then 1 else x * (x ** (n - 1))
```

Now `2 ** 10` returns `1024` — the `**` operator can be used in infix style just like any built-in operator.

Generic extensions. Extensions can be parameterised, which makes them work for any type:

```
extension [T](xs: List[T])
  def second: T = xs.tail.head
```

This adds a `second` method to lists of any element type:

```
List(1, 2, 3).second        // 2
List("a", "b", "c").second  // "b"
```

Collective extensions. When several extension methods share the same receiver type, they can be grouped under a single `extension` block:

```
extension (s: String)
  def shout: String = s.toUpperCase + "!"
  def whisper: String = "(" + s.toLowerCase + ")"
  def reversed: String = s.reverse

"hello".shout      // "HELLO!"
"WORLD".whisper    // "(world)"
"abc".reversed     // "cba"
```

Extension methods will give us a clean way to define operations on abstract types: instead of putting methods inside a class, we will define the type abstractly and add operations through extensions. This pattern combines naturally with the *opaque types* mechanism that the encapsulation lecture introduces.

Summary

This detour has introduced four Scala features that will be used in the encapsulation lecture:

- **Classes** group fields and methods into a single unit, with a primary constructor in the class header. Fields tagged `val` or `var` are accessible; otherwise they are private.
- **Singleton objects**, introduced with `object`, are unique automatically-created instances of an anonymous class. They host static-like definitions: functions, constants, factories.
- **Companion objects** are singleton objects that share their name with a type definition (a class, a trait, an `enum`, or an opaque type). They have privileged access to the type's private members and are the natural place for factory methods. The `apply` method enables the no-`new` construction syntax.
- **Case classes and sealed traits** are the standard Scala mechanism for defining algebraic data types — they provide automatic factory methods, pattern matching, structural equality, and exhaustiveness checking.
- **Extension methods** add methods (and operators) to existing types without modifying their definitions. They can be generic and grouped collectively.